

Assignment: Banking Fraud Detection and Customer Notification System

Problem Statement

Your task is to design and deploy a **cloud-native banking application** that detects potentially fraudulent transactions in real time and notifies customers. The application must simulate a banking environment where customers perform transactions, and the system evaluates risk factors before allowing or flagging them.

The project has **two core components**:

1. Transaction Processing & Fraud Detection (ECS Service)

- Build a REST API backend using **Node.js / Python** and deploy it on **Amazon ECS (Fargate)**.
- The API should accept transaction details such as:
 - `account_id`, `amount`, `transaction_type` (withdrawal, deposit, transfer), `location`, `timestamp`.
- Implement simple fraud detection rules such as:
 - Unusually large withdrawal amount.
 - Transactions from different geographic regions within a short time window.
 - Too many failed login attempts before a transaction.
- If the transaction looks **valid**, mark it as “approved.”
- If the transaction looks **suspicious**, mark it as “flagged” and push an event to an **AWS SQS queue**.

2. Customer Notification & Alerting (AWS Lambda Function)

- Write an **AWS Lambda function** that listens to the SQS queue.

- If a transaction is flagged as fraudulent, send a notification (e.g., email via **Amazon SES**, or log an alert message).
 - Store flagged transactions into a **DynamoDB table** for record-keeping and analytics.
-

Deployment & Infrastructure Requirements

- Use **AWS CDK or Terraform** to provision infrastructure.
 - **ECS (Fargate)** for the transaction API service.
 - **AWS Lambda** for processing flagged transactions.
 - **Amazon SQS** for decoupling ECS and Lambda.
 - **Amazon DynamoDB** for storing fraud logs.
 - **Amazon SES** (or SNS) for sending customer alerts.
 - **ALB (Application Load Balancer)** to expose the ECS service API securely.
 - Secure the infrastructure using **IAM roles and policies**.
-

Deliverables

1. **Application Code:**
 - REST API backend code (transaction processing & fraud detection).
 - Lambda function code (fraud notifications & database updates).
2. **Infrastructure as Code (IaC):**
 - CDK/Terraform code to set up ECS, Lambda, SQS, DynamoDB, SES/SNS, and networking components.
3. **Documentation:**

- Architecture diagram.
- Explanation of fraud detection logic.
- Steps to deploy the solution.
- API usage instructions (endpoints, request/response format).

Bonus Challenges 🚀

- Implement **JWT authentication** for the API to simulate customer logins.
- Integrate **Amazon Kinesis Firehose + S3** for streaming flagged transaction data for future ML analysis.
- Add **CloudWatch Alarms** to alert if fraud detection volume spikes.
- Make the application **multi-tenant** to support multiple banks with separate fraud thresholds.